

T4 de SisOp

Sistema de Arquivos sofs

Prof. Lucas Mello Schnorr
schnorr@inf.ufrgs.br

Este trabalho implementa o sistema de arquivos sofs. O esqueleto fornecido contém a camada de bloco (sofs-block) completamente implementada, as funções de gerência do sistema de arquivos (sofs_format, sofs_mount, sofs_umount, sofs_identify) e funções auxiliares prontas para alocação e liberação de blocos de dados e i-nodes (alloc_data_block, free_data_block, alloc_inode, free_inode). A tarefa do grupo é implementar as funções de arquivo, diretório e link marcadas com TODO em sofs.c. Ao concluir, o grupo será capaz de: 1/ Implementar criação, abertura, leitura, escrita e remoção de arquivos sobre uma camada de blocos e bitmaps; 2/ Percorrer o diretório raiz de estrutura linear; 3/ Criar vínculos simbólicos e estritos (softlinks e hardlinks).

1 Descrição do Sistema de Arquivos

O disco virtual t2fs_disk.dat é dividido em setores de 256 bytes. O primeiro setor (0) é o MBR, que descreve até quatro partições. Cada partição formatada com sofs_format recebe, em ordem: (1) o superbloco no bloco 0; (2) freeBlocksBitmapSize blocos de bitmap de blocos de dados; (3) freeInodeBitmapSize blocos de bitmap de i-nodes; (4) a área de i-nodes com 10% dos blocos da partição (arredondado para cima); (5) os blocos de dados restantes. Arquivos são referenciados por um diretório raiz de nível único, cujas entradas têm tipo, nome (até 50 caracteres alfanuméricos) e número de i-node. Cada i-node possui dois ponteiros diretos, um de indireção simples e um de indireção dupla. Softlinks ocupam sempre um bloco e armazenam o nome do arquivo alvo. Hardlinks apontam para o mesmo i-node e incrementam o campo RefCounter.

2 Material Fornecido

O esqueleto sofs.c é disponibilizado com: as funções de gerência já implementadas (sofs_format, sofs_mount, sofs_umount, sofs_identify); as funções auxiliares de criação/destruição de blocos e i-nodes (alloc_data_block, free_data_block, alloc_inode, free_inode); os protótipos das funções de arquivo, diretório e link com comentários TODO. O módulo sofs-block (já completo) provê read_block / write_block. As bibliotecas pré-compiladas apidisk.o e bitmap2.o proveem, respectivamente, read_sector / write_sector e a API de bitmap (openBitmap2, closeBitmap2, getBitmap2, setBitmap2, searchBitmap2). **O grupo deve implementar exclusivamente as funções marcadas com TODO em sofs.c.** Também é fornecido o programa de exemplo sofs/src/exemplo.c, que demonstra o uso das funções já implementadas e serve de inspiração para os testes que o grupo deverá criar. O exemplo exercita sofs_format, sofs_mount e sofs_umount, e contém blocos comentados que ilustram como testar sofs_create, sofs_write, sofs_read, sofs_opendir / sofs_readdir, sofs_sln e sofs_hln. Para compilá-lo:

```
# gera bin/exemplo (requer -m32)
make exemplo
```

Os objetos pré-compilados apidisk.o e bitmap2.o são binários i386 (32 bits). Em host 64 bits é necessário ter gcc-multilib instalado; o Makefile já passa -m32 para a linkagem.

3 Roteiro de Implementação

As funções a implementar estão em sofs.c com comentários de orientação em cada stub. Recomenda-se a seguinte ordem:

- sofs_create(filename): usa alloc_inode para reservar um i-node; procura uma entrada de diretório livre no bloco de dados do diretório raiz (i-node 0) e preenche o registro com TYPEVAL_REGULAR, o nome e o número do i-node; adiciona o arquivo à tabela de arquivos abertos e retorna o handle. Se o arquivo já existir, remove seus blocos de dados e zera o tamanho.
- sofs_delete(name): encontra o registro de diretório de name; percorre todos os ponteiros do i-node e libera os blocos alocados com free_data_block; libera o i-node com free_inode; invalida o registro de diretório (TYPEVAL_INVALIDO).
- sofs_open(name) / sofs_close(handle): localiza o registro de diretório, aloca/libera uma entrada na tabela de arquivos abertos (máximo 10 simultâneos), inicializa/zera o ponteiro corrente.
- sofs_read(handle, buffer, size) / sofs_write(handle, buffer, size): traduz o ponteiro corrente em número de bloco e deslocamento; usa read_block / write_block para transferir os dados; em escrita, aloca novos blocos com alloc_data_block se necessário; avança o ponteiro corrente.
- sofs_opendir / sofs_readdir / sofs_closedir: percorre as entradas do bloco de dados do i-node 0 (diretório raiz), retornando apenas entradas com TypeVal != TYPEVAL_INVALIDO.
- sofs_sln(linkname, filename): cria um softlink com alloc_inode + alloc_data_block; grava o string filename no bloco de dados; preenche o registro de diretório com TYPEVAL_LINK.
- sofs_hln(linkname, filename): localiza o i-node do arquivo filename; cria um novo registro de diretório apontando para o mesmo i-node; incrementa RefCounter no i-node.

4 Restrições Técnicas

Restrição	Detalhe
Linguagem	C (padrão C99)
Modificações permitidas	Nas funções TODO de sofs.c
Compilação	Conforme Makefile fornecido
Proibido	Alterar sofs-block.[ch]
Proibido	Alterar structs em sofs-block.h
Arquivos abertos	Máximo 10 simultaneamente

5 Entregáveis

Entrega via Moodle no link correspondente:

- sofs.c com todas as funções TODO implementadas.
- Relatório TXT em UTF-8 (sem \r) com o realizado.
- Exemplos funcionais e corretos do uso do FS.

6 Critérios de Avaliação

Critério	Peso
sofs_create / sofs_delete corretos	20%
sofs_open / sofs_close corretos	20%
sofs_read / sofs_write corretos	20%
sofs_opendir / sofs_readdir corretos	20%
sofs_sln correto (softlink)	10%
sofs_hln correto (hardlink)	10%

7 Dicas

- Use o exemplo fornecido como ponto de partida: sofs/src/exemplo.c já compila e roda com as funções

implementadas. Descomente os blocos numerados progressivamente conforme você implementa cada função, e execute `make exemplo` para verificar o comportamento. Adapte o exemplo para cobrir casos de borda relevantes ao seu grupo.

- **Leia `sofs-block.h` completamente** antes de começar: as estruturas `sofs_mbr`, `sofs_superbloco`, `sofs_inode` e `sofs_record` são os tijolos de toda a implementação.
- **Use as funções auxiliares fornecidas:** `alloc_data_block` já chama `searchBitmap2` + `setBitmap2` e zera o bloco; não reimplente essa lógica.
- **O i-node 0 é o diretório raiz:** após `sofs_mount`, o i-node 0 deve estar marcado como ocupado no bitmap de i-nodes e seus blocos de dados contêm os registros `sofs_record`.
- **Ponteiros de indireção:** implemente primeiro os ponteiros diretos (`dataPtr[0]` e `dataPtr[1]`) e depois o `singleIndPtr`; o `doubleIndPtr` exige dois níveis de bloco de índice.
- **Softlink vs hardlink:** em `sofs_open`, se `TypeVal != TYPEVAL_LINK`, leia o bloco de dados do i-node para obter o nome do alvo e faça uma nova busca no diretório (até um limite de indireções para evitar ciclos).

8 Referências

- Silberschatz, Cap. 11 (Implementação do Sistema de Arquivos), Secs. 11.1–11.4.
- Tanenbaum, Cap. 4 (Sistemas de Arquivos), Secs. 4.3 e 4.4.